学习情境 L07：NL2SQL 管线全景—从一句话到一条安全 SQL（6 学时）

一、学习情境基本信息

项目	内容
学习领域	LF3 智能查询服务开发：NL2SQL 与知识引擎
讲次	第 07 讲（第 7 周）
学时	6 学时（校内理实一体 4 学时 + 企业站实践 2 学时）
布鲁姆主层级	**应用**（记忆、理解、分析渗透）
双元场地	校内：理实一体机房；企业站：DAP-v2 集群，洞察 Agent 服务 `insight-agent`
前置	L05 数仓已灌 105.9 万订单；L06 网关读写路径理解；LiteLLM→qwen2.5:14b 服务可达
企业站 PR 任务	**PR-07**：跑通 3 条指定问句，逐条交付"问句 → 生成 SQL → 行数/结果 → 护栏记录"证据链
课后费曼任务	把五步管线讲成"外卖下单"类比，讲给完全不懂 AI 的听众并录 3 分钟音频

二、任务描述（客户委托单）

委托单编号：DAP-NL2SQL-07

业务方在门户输入一句自然语言："2026 年5 月销售额最高的10 个品类"。洞察 Agent 组委托你们：全程追踪这句话如何变成一条安全可执行的 SQL——跑通五步管线并留存每步产物；解剖评测集 Q17 题从"错选表"到修复的完整过程；亲手触发 LLM 输出的 JSON 解析容错与自校验重试路由，最后交付 3 条问句的 SQL 与行数证据链，证明"答案不是编的"。

三、学习目标

1. 【记忆】按序复述 NL2SQL 五步管线：意图理解 → Schema Linking（列级反向索引）→ LLM 生成（LiteLLM→qwen2.5:14b，温度 0.1，JSON 模式）→ 自校验（列归属 + 重试≤2）→ 沙箱执行（四层护栏）。
2. 【理解】解释列级反向索引为什么能把 prompt 从"全库 schema"压缩到"表 X 字段清单"，以及对准确率与 token 成本的双面收益。
3. 【应用】运行管线 CLI 并留存五步产物文件（intent / schema_link / sql / check / result），解释每个文件中关键字段的来源。
4. 【分析】定位 Q17（"2026 年 5 月销售额最高的 10 个品类"）错选表根因，说明"同义词映射 + 分层白名单"修复方案并复现修复前后对比。

5. 【理解】说明 JSON 解析容错的三级降序策略与自校验失败后的重试路由（重试≤2 次后转人工/模板兜底），并说出对应指标名。

四、教学重点与难点

教学重点

1. 五步管线逐环产物与数据流：每一环的输入、输出、失败出口（缺一环讲不清整链）。
2. Q17 真实错题的根因分析：Schema Linking 层的"列→表"映射错误如何污染下游所有环节。

教学难点

1. 学生普遍低估 Schema Linking 的价值，认为"把全库 schema 塞进 prompt 也行"。

突破策略：现场算账 + 干扰实验——① token 账：全库 5 层 40+ 表 ≈ 12k token/次 vs 命中 2 表 ≈ 800 token/次（成本 15 倍差）；② 干扰账：给 qwen2.5:14b 看全部表时现场复现"错选 ODS 表"（Q17 旧版行为），只看白名单表时稳定正确——用同一模型两次结果对比，让"少即是多"可感。

2. "自校验放在 LLM 之后"的必要性（LLM 会幻觉出不存在或不在白名单的列）。

突破策略：幻觉注入演示——构造 prompt 让模型生成 `dwd_order_items.profit_margin`（表中不存在），观察 validator 拦截并触发重试路由，展示 `check` 产物里的 `violations` 字段。

五、教学准备

类别	内容
环境	`insight-agent` 服务与 CLI 可用；TiDB 数仓就绪（表：`ods_orders`、`dwd_orders`、`dwd_order_items`、`dim_products`、`dws_sales_daily` 等）；LiteLLM 网关路由 `qwen2.5:14b`（`temperature=0.1`，`response_format=json`）
代码	`services/insight-agent/app/nl2sql/`：`intent.py` / `schema_link.py` / `generator.py` / `validator.py` / `executor.py`；索引快照 `evals/fixtures/schema_index_v0.jsonl`（Q17 修复前版）与 `services/insight-agent/resources/schema_index.jsonl`（现行版）
评测	`evals/nl2sql_evalset.jsonl`（24 题，现基线 accuracy=0.917，bootstrap 95%CI [0.792, 1.000]）；`evals/run.py`
工具	curl、jq、mysql 客户端（复核行数）；任务单含"证据链粘贴区"
分组	沿用结对；每组认领 1 条"必翻车问句"用于 L08 铺垫（如"订单号 O 开头怎么查"）

六、教学过程设计（六步法：资讯 30′/计划 30′/决策 20′/实施 90′/检查 40′/评价+学生讲堂 30′）

6.1 资讯（30′）

【教师活动】

1. 以 Q17 问句开场，先演示最终效果（门面在最后揭穿）：

```
python -m services.insight_agent.nl2sql run \
  --q "2026 年 5 月销售额最高的 10 个品类" --trace out/q17/
// out/q17/3_sql.json（产物示例）
{"sql": "SELECT p.category, SUM(i.pay_amount) AS sales_amt
        FROM dwd_order_items i JOIN dim_products p ON i.product_id = p.product_id
        WHERE i.order_ts >= '2026-05-01' AND i.order_ts < '2026-06-01'
        GROUP BY p.category ORDER BY sales_amt DESC LIMIT 10",
 "tables": ["dwd_order_items", "dim_products"], "confidence": 0.86}
```

2. 板书五步管线数据流（含每步失败出口）：

```
问句 ─①意图理解─► intent{domain, time_range, metrics, topn}
      ─②Schema Linking（列级反向索引）─► 候选表 + "表 X 字段清单" prompt 注入
      ─③LLM 生成（LiteLLM→qwen2.5:14b, temp=0.1, JSON 模式）─► {sql, tables}
           ├─ JSON 解析失败 → 剥围栏/正则提段/重生成 1 次（parse_fail_total 计数）
       ─④自校验：列归属（SQL 中每列 ∈ 候选表字段清单？）+ 只读白名单预检
            ├─ 违规 → 带违规原因重试路由回 ③（≤2 次）→ 仍败 → 模板/人工兜底
      ─⑤沙箱执行（四层护栏：AST 只读白名单 / 强制 LIMIT+超时 / 租户表授权 / 高危审批）
─► 行集 + 行数证据
```

3. 讲解两个关键设计动机：① 反向索引 = "列名/同义词 → 表.列"的倒排，把 12k token 压到 ~800；② 温度 0.1 + JSON 模式 = 让同一个问句的输出尽量确定、可解析（评测集可回归的前提）。

4. 预告评测基线：24 题 91.7%（22/24），CI 下界 79.2%——"你们今天要亲手抓到那 2 题中的一题的历史版本"。

【学生活动】 复画管线图并给每步标注"失败出口"；在任务单记录 token 账与温度/JSON 模式的两个"为什么"。

【教学方法】 演示导入 + 板书数据流 + 追问。

【设计意图】 先见结果再见骨架，动机（成本、确定性）先行，避免管线沦为名词背诵；失败出口为实施环节的容错实验建立预期。

6.2 计划（30′）

【教师活动】 指导各组排实施顺序与证据采集计划（三实验 + 一复盘）：

序	实验	产物	时间盒
E1	五步全流程追踪（自选问句 + Q17）	out/*/** 五件产物	25′
E2	Q17 错选表复现与修复对比	修复前后 2_schema_link/3_sql 截图	25′
E3	JSON 容错 + 幻觉列重试路由注入	fuzz 输出 + check 违规字段	25′

E4	行数复核（mysql 直查对账）	对账 SQL 与行数	15′

【学生活动】 结对定自选问句（避开 Q17）；预写 E2 的"预期差异"（旧索引会把"销售额"链到哪张表？）；分配 Driver/Navigator 轮换点。

【教学方法】 实验矩阵计划法。

【设计意图】 把 90′ 实施拆成可验收的四个实验，每个实验都有对照物（前后/两侧），训练"对比取证"习惯。

6.3 决策（20′）

【教师活动】 审核各组问句是否覆盖不同环节风险（时间解析 / 多表 JOIN / 聚合维度各至少一题）；统一安全口径：所有跑数走 `dap_app` 只读账号；`--trace` 目录按问句 ID 归档避免覆盖。

【学生活动】 微调问句清单；确认归档结构 `out/{qid}/1_intent.json … 5_result.csv`。

【教学方法】 清单审核 + 教师放行。

【设计意图】 问句覆盖度直接决定证据链的教学价值；归档规范为评测回归（L15）留档。

6.4 实施（90′）

子步骤 1（25′）：E1 全流程追踪与产物解读

```
python -m services.insight_agent.nl2sql run \
  --q "2026 年 5 月销售额最高的 10 个品类" --trace out/q17/
cat out/q17/1_intent.json
# {"domain":"sales","time_range":{"start":"2026-05-01","end":"2026-06-01"},
#  "metrics":["销售额"],"topn":10,"intent":"rank"}
cat out/q17/2_schema_link.json
# {"candidates":["dwd_order_items","dim_products","dws_sales_daily"],
#  "prompt_tables":{"dwd_order_items":"id,order_id,product_id,qty,pay_amount,order_ts,…",
#                   "dim_products":"product_id,sku,category,brand,…"},
#  "index_hits":["销售额→dwd_order_items.pay_amount","品类→dim_products.category"]}
```

任务单必答：②的 `prompt_tables` 里为什么没有 `ods_orders`？（答：索引白名单只收 DWD/DWS/DIM/ADS，ODS 不对外。）

子步骤 2（25′）：E2 Q17 错选表复现与修复对比（本讲核心）

```
# 用修复前索引快照复现错选表：
python -m services.insight_agent.nl2sql run --trace out/q17_old/ \
  --q "2026 年 5 月销售额最高的 10 个品类" \
  --index evals/fixtures/schema_index_v0.jsonl
cat out/q17_old/2_schema_link.json
# 旧版索引把 "销售额" 命中到 ods_orders.total_amount（字段名最直白，但 ODS 含运费且不该对外）
cat out/q17_old/3_sql.json | jq -r .sql
# SELECT … FROM ods_orders …            ← 错选表：口径错（含运费）+ 层次错（ODS 暴露）
```

修复三步（对照现行 `resources/schema_index.jsonl`）：

1. **分层白名单**：Schema Linking 只在 DWD/DWS/DIM/ADS 层的列上建倒排；
2. **同义词映射**：`销售额/营业额/营收 → dwd_order_items.pay_amount`、`销售额(日汇总场景) → dws_sales_daily.sale_amount`；
3. **评测回归**：

```
python -m evals.run --set evals/nl2sql_evalset.jsonl --only Q17   # PASS
python -m evals.run --set evals/nl2sql_evalset.jsonl              # 24 题 0.917 不回退
```

子步骤 3（25'）：E3 JSON 容错与幻觉列重试路由

```
# ① JSON 容错三级降序（注入模型常见劣化输出）：
python -m services.insight_agent.nl2sql.fuzz_json \
  --case fence --case trailing-comma --case truncated
# 输出示例：
# fence           → strip ```json 围栏后 parse OK (lenient=1)
# trailing-comma  → 剥尾逗号后 parse OK (lenient=1)
# truncated       → parse FAIL → 重生成 1 次 → parse_fail_total{reason="truncated"} +1

# ② 幻觉列注入：诱导生成不存在的 profit_margin 列
python -m services.insight_agent.nl2sql run --trace out/hallu/ \
  --q "2026 年 5 月各品类利润率排名前 10"      --index evals/fixtures/index_with_profit_hint.jsonl
cat out/hallu/4_check.json
# {"passed":false,"violations":["column dwd_order_items.profit_margin not in table schema"],
#  "retry":1,"route":"regenerate_with_violations"}
# 重试 ≤2 次仍败 → 兜底：返回"该指标未在数仓口径中定义，建议口径：利润率=…（未落库）"
```

子步骤 4（15'）：E4 行数对账（证明"答案不是编的"）

```
mysql -h 10.167.2.175 -P 4000 -u dap_app -p -e "
SELECT COUNT(*) FROM (SELECT p.category, SUM(i.pay_amount) amt
  FROM dwd_order_items i JOIN dim_products p ON i.product_id=p.product_id
  WHERE i.order_ts>='2026-05-01' AND i.order_ts<'2026-06-01'
  GROUP BY p.category ORDER BY amt DESC LIMIT 10) t;"
# 与 out/q17/5_result.csv 行数一致（=10）→ 对账通过
```

【教师活动】 巡回；在 E2 处集中演示一次"索引差异 diff"（`diff <(jq -S . old) <(jq -S . new)`）把"修复"从概念变成两行可见的索引条目；收集各组幻觉列案例（哪个列名被幻觉最多——常见 `profit_margin`/`user_name`）。

【学生活动】 按实验矩阵执行并粘贴证据；每完成一实验在任务单打勾并记录偏差（如 fuzz 某 case 与预期不符，原样记录留讨论）。

【教学方法】 对比实验法 + 注入式测试（fuzz/幻觉注入）。

【设计意图】 E2 是全课"用数据说话"的样板：错误可复现、修复可 diff、回归有指标；E3 让学生站在管线作者视角制造故障，理解容错代码存在的理由（而非"别人写好的黑盒"）。

6.5 检查（40'）

【教师活动】 按证据链四查验收：① 五步产物齐全且字段能解释；② Q17 前后对比两截图 + 索引 diff；③ fuzz 三 case 输出 + check 违规字段；④ 对账行数一致。追问两组："如果 qwen2.5:14b 换成 72b，哪些环节可能要重调？"（答：JSON 模式稳定性↑可简化容错；幻觉列概率↓但 validator 不能删——护栏与模型能力解耦）。

【学生活动】 组间交换证据链互查（重点查 5_result.csv 行数与对账 SQL 是否真跑过）；登记缺项并现场补跑或如实标注。

【教学方法】 证据链核验 + 迁移追问。

【设计意图】 "护栏与模型能力解耦"是本讲最重要的架构判断，用换模型思想实验检验学生是否真正理解各环节存在的原因。

6.6 评价 + 学生讲堂（30’）

【教师活动】 按第八节评分；学生讲堂："我把五步管线讲成外卖下单"（随机抽 3 组各 3 分钟：意图=读懂你想吃啥、Schema Linking=只带相关餐厅的菜单、生成=后厨照单做、自校验=出餐前核对有没有点了没有的菜、沙箱执行=通过食安检查才出餐）；布置 L08 预习：思考"为什么 Q17 对了，'订单号 O 开头怎么查'这类问句还是会翻车"。

【学生活动】 类比讲解（类比必须保真：映射错一处扣分）；听众互评"类比是否帮理解而非误导"；完成自评三问。

【教学方法】 费曼讲堂 + 类比保真互评。

【设计意图】 类比是对理解的最强压缩测试——映射错（如把自校验说成"做错了重做"漏掉列归属）立即暴露理解空洞；预习问题直通 L08 混合检索。

七、企业站实践任务卡（2 学时）

任务卡 PR-07：三条问句证据链交付

项目	内容
任务目标	跑通 3 条指定问句，交付"问句→SQL→行数/结果→护栏记录"完整证据链，并修复其中暴露的一个索引缺口
前置条件	E1-E4 校内完成；dap_app 只读账号可用
指定问句	① "2026 年第二季度各区域的订单量排名" ② "复购客户（≥2 单）贡献的销售额占比" ③ 各组自选 1 条（需覆盖时间筛选或 JOIN 之外的第三种风险：模糊条件/口径词）

步骤：

```
for q in "2026 年第二季度各区域的订单量排名" "复购客户（≥2 单）贡献的销售额占比" "<自选>"; do
  python -m services.insight_agent.nl2sql run --q "$q" --trace "out/pr07/$(echo $q|md5sum|cut -c1-6)/"
done
# 每条问句交付：
#  1) 5 步产物目录（1_intent…5_result）
#  2) 对账：mysql 用同 SQL 复核行数/金额，截图
#  3) 若自校验有 retry≥1：解释违规原因与重试后行为
#  4) 若问句失败：定位失败在第几步 + 根因（索引缺口/口径未定义/模型解析），能修索引的修，不能修的写入"未解决清单"
```

交付物（PR 内容）： docs/07-项目资源/g{N}/L07-证据链.md（3 问句 × 4 要素表格化）、out/pr07/ 产物、索引补丁（若有）。**必须含**"失败与未解决清单"栏。

评审标准（企业导师三查）： 查链（SQL 与结果能互相对账）；查真（产物时间戳/行数可复核，禁止手改 jsonl 产物）；查根因（失败项定位到管线步骤，不允许写"模型不行"了事）。

双导师签栏	意见	签名/日期
校内导师		
企业导师		

八、评价设计（任务单评分表）

维度	权重	90-100 分标准	75-89 分标准	60-74 分标准	<60 分标准
管线复述（资讯）	15%	五步 + 失败出口全对	五步对	漏一步	靠提示
E1 产物解读（实施）	20%	五件产物字段全部能解释	4 件	3 件	未跑通
E2 根因分析（实施）	25%	复现+索引 diff+回归三证齐全，能讲清分层白名单逻辑	缺回归	缺 diff	未复现
E3 容错实验（实施）	20%	三 case + 幻觉注入全做且解释路由	缺一项	仅跑通	未做
E4 对账（实施）	10%	3 条问句全对账一致	2 条	1 条	未对账
学生讲堂（评价）	10%	类比映射全保真	一处失真	多处失真	讲不出

红线：手改产物文件伪造"正确 SQL"或对账截图与实际行数不符，本讲 0 分重做（HSA）。

九、课后费曼任务

项目	内容
讲给谁	完全不懂 AI 与数据库的家人/朋友
讲什么	用"外卖下单"类比讲五步管线，录 3 分钟音频：每一步映射必须保真（意图=点单备注、Schema Linking=只带相关菜单、生成=后厨备菜、自校验=出餐前核对、沙箱=食安检查），并回答对方至少 1 个追问（如"为什么不让 AI 直接连数据库跑？"——答：幻觉列/口径错/权限，四层护栏的存在理由）
自评三问	① 我的类比里哪一步映射最勉强，听众听懂了吗？② 我能不能不用任何类比，直接用三句话讲清"为什么先 Schema Linking 再生成"？③ 如果业务问"利润率"而数仓没有这列，我的系统会发生什么——我能把整条失败路径背出来吗？

十、教学反思栏

反思维度	记录（课后填写）
"全 schema 也行"误区破除效果	

E2 复现成功率与卡点	
幻觉列高频名单（收集数据）	
类比讲解中的典型失真	
未解决清单汇总（转 L08 议题）	